If, however, retailers' stock is dynamic, and items can go out of stock in one place or start being re-stocked in another, then our current model won't work, and we to add a tiny change to our tree nodes and our search method. But we will talk about these details later in this chapter: first, let's see how we can write a solution to the basic version of our e-commerce problem.

11.1.1 Sketching a Solution

Initially we will make some assumptions to simplify our scenario: assume all the shops have all the products in stock at all times. Figure 11.2 shows a possible workflow for order processing under this scenario; of course, this is not what would happen in reality, but simplifying a problem usually helps us sketching a first solution upon which we can iterate, introducing one by one the constraints found in real situations, and reasoning about how they influence our applications and how we can cope with them:

- Shops have dynamic inventory, so we need to check that the closest shop to a customer actually sells an item and has it in stock.
- Delivery costs and other considerations might suggest us to prefer a further shop to the closest one.
- If we need to make HTTP calls as part of our workflow, we need extra care to avoid network pitfalls, and we might have to carefully select our algorithm to cope with issues like fault tolerance and timeouts.

Listing 11.1 The Shop class